

```
Loading package uuid-1.3.8 ... linking ... done.
Loading package postgresql-simple-0.4.10.0 ... linking ... done.
```

You can verify the created table from the *psql* prompt:

```
test=# \d social.users
      id | integer |
      fname | character varying(80) |
      lname | character varying(80) |
```

You can also list the databases in the PostgreSQL server using the *query_* function as illustrated below:

```
{-# LANGUAGE OverloadedStrings #-}
import Database.PostgreSQL.Simple
main :: IO ()
main = do
  conn <- connect defaultConnectInfo
    { connectUser = "postgres"
    , connectPassword = "postgres123"
    , connectDatabase = "test"
    }
  databases <- query_ conn "SELECT datname FROM pg_database"
  print (databases :: [Only String])

  close conn
```

Executing the above code in GHCi produces the following output:

```
$ ghci show.hs
GHCi, version 7.6.3: http://www.haskell.org/ghc/ :? for help
Loading package ghc-prim ... linking ... done.
Loading package integer-gmp ... linking ... done.
Loading package base ... linking ... done.
[1 of 1] Compiling Main             ( show.hs, interpreted )
Ok, modules loaded: Main.
```

```
ghci> main
[Only {fromOnly = "template1"},Only {fromOnly =
"template0"},Only {fromOnly = "postgres"},Only {fromOnly =
"test"}]
```

You can now insert a record into the databases using the *execute* function:

```
execute conn "insert into social.users (id, fname, lname) values
(?, ?, ?)" ["1" :: String, "Edwin" :: String, "Brady" :: String]
```

After executing the above code, you can verify the database entry from the *psql* prompt:

```
test=# select * from social.users;
 id | fname | lname
```

```
-----+-----+-----
  1 | Edwin | Brady
(1 row)
```

You can also do batch inserts using the *executeMany* function. For example:

```
executeMany conn "insert into social.users (id, fname, lname)
values (?, ?, ?)" [(2 :: String, "Simon" :: String, "Marlow" ::
String), (3 :: String, "Ulf" :: String, "Norell" :: String)]
```

After running the above code, you can check the newly added rows in the database from the *psql* command-line tool:

```
test=# select * from social.users;
 id | fname | lname
-----+-----+-----
  1 | Edwin | Brady
  2 | Simon | Marlow
  3 | Ulf   | Norell
(3 rows)
```

You can also change a record entry using the *UPDATE* statement as shown below:

```
execute conn "update social.users SET lname = 'Peyton Jones'
where fname = 'Simon'" ()
```

The corresponding entry is updated as seen from the *psql* prompt:

```
test=# select * from social.users;
 id | fname | lname
-----+-----+-----
  1 | Edwin | Brady
  2 | Ulf   | Norell
  3 | Simon | Peyton Jones
(3 rows)
```

It is recommended that you catch exceptions when running database commands. Consider the following example, where the number of arguments passed does not match with what is expected:

```
{-# LANGUAGE OverloadedStrings #-}

import Database.PostgreSQL.Simple
import Control.Exception
import GHC.Int

main :: IO ()
main = do
  conn <- connect defaultConnectInfo
    { connectUser = "postgres"
```

Continued on page ...39